Laboratory 1: Short project

Docker, Compose and Swarm — high availability edition

Marc De Rialp David Franco Omar Antonio Cornejo David Madrid
Victor Yakovlev

Cloud Computing and Distributed Systems
MSc in Machine Learning and Cybersecurity for Internet Connected Systems, UPC

October 2026

Abstract

We containerised the digital infrastructure of a company with Docker: a shared stack (web front, certificates, shared disk), a reviews application with its database, cache, mail relay and backups, a monitoring platform, and the Moodle learning management system. The same project runs in two environments: a local one on a single machine (`compose.yml`) and a production one on a Docker Swarm of seven nodes (`production.yml`), in which every user-facing service is replicated, the database and the cache are clustered and fail over by themselves, and the backups are encrypted and kept on a node of their own. The images are published in Docker Hub, so deploying only needs to pull and start them. The report describes the design, the deployment, the management of data and backups, the monitoring, the test cases and their results.

Contents

1 Introduction **3**

2 Architecture **3**
2.1 Nodes . . . 3
2.2 Services . . . 3

3 Images and design decisions **4**

4 Scalability and high availability **6**
4.1 Replicas . . . 6
4.2 Failures . . . 6
4.3 Updates . . . 6

5 Deployment **6**

6 Access, users and passwords **7**
6.1 Web addresses . . . 7
6.2 Users and passwords . . . 7

7 Data management and backups **7**

8 Observability **8**

9 Security **9**

10 Test cases **9**
10.1 Local environment . . . 9
10.2 Production environment . . . 10

11 Results **11**
11.1 Local environment . . . 11
11.2 Production environment (`production.yml`) . . . 11

12 Limitations **11**

13 Conclusions **12**

1 Introduction

We containerised the infrastructure of Figure 1 of the statement: the shared services (*basic*), a company application (*app*), an observability platform (*stats*) and Moodle (*atenea*). As the core application we wrote a review site in Flask: people leave a comment with a rating from 1 to 5, see the distribution and the latest reviews, each new comment sends a mail through our SMTP relay and an automatic feeder collects ratings published in other web pages.

This version goes beyond a single working deployment: every user-facing piece has at least two replicas on different nodes, the application database and the cache are clustered, the backups are encrypted and kept on a node of their own, and every service is monitored. Everything is deployed with one `docker compose up` (local) or one `docker stack deploy` (production), with the images already published in Docker Hub.

The repository contains `compose.yml` (local), `production.yml` (swarm), one folder per image with its Dockerfile and files, `scripts/` (node labels, publication, simulated swarm and restore) and this document. The local environment has no TLS and uses named volumes, but it contains the same clusters (database, cache) as production, so they can be tested on one machine.

2 Architecture

2.1 Nodes

The swarm has seven nodes; the placement is done with the node label `role` (and `slot` for the two app nodes, to keep a primary and its replica apart).

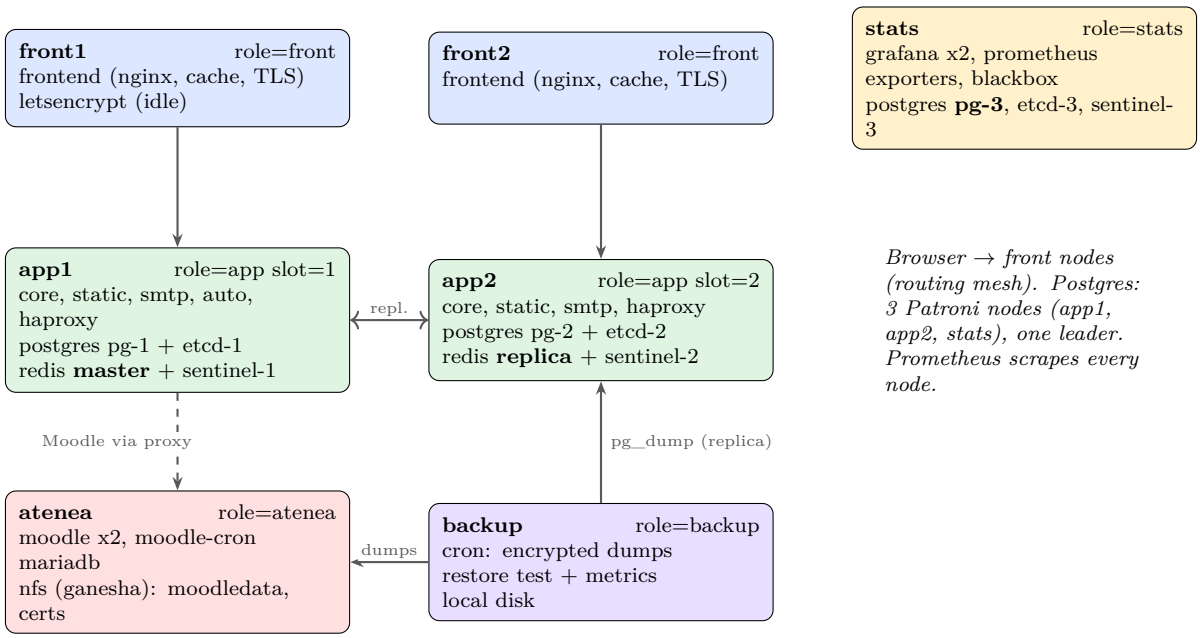


Figure 1: Nodes and services (simplified). The routing mesh accepts the traffic on both front nodes; `backup` pulls from a replica, from MariaDB and from the NFS volume.

2.2 Services

Table 1 maps the boxes of the diagram of the statement to our services, the node where they run and their replicas. Each stack is an encrypted overlay network and a service only joins the networks it needs: the databases are not in the `proxy` network, so the frontend can not reach them.

Public names in production: `app.<domain>`, `lms.<domain>` and `grafana.<domain>`; with the default `DOMAIN=localhost` they are `app.localhost`, `lms.localhost` and `grafana.localhost` on

Stack	Service	Nodes	Repl.	Description
basic	`frontend`	front	2	nginx: only public service (80/443), cache, TLS, proxy by subdomain
	`letsencrypt`	front	1	certbot; idle with a local domain (self-signed certificate)
	`nfs`	atenea	1	NFSv4 server (NFS-Ganesha) inside the stack, port 2049 in host mode
app	`core`	app	2	Flask + gunicorn, reviews
	`static`	app	2	Apache httpd, `/static/`
	`pg-1..3`	app1, app2, stats	3	PostgreSQL 16 + Patroni (leader + 2 replicas)
	`etcd-1..3`	app1, app2, stats	3	consensus store of Patroni
	`db`	app	2	HAProxy: `db:5432` leader, `db:5433` replicas
	`redis-1`, `redis-2`	app1, app2	1+1	cache: master and replica
	`sentinel-1..3`	app1, app2, stats	3	failover of the cache (quorum 2)
	`smtp`	app	2	Postfix relaying the mails (to Mailpit by default)
	`mailpit`	app	1	mail catcher with a web page (`mail.localhost:8080`)
	`auto`	app	1	feeder of external ratings
	`backup`	backup	1	encrypted dumps + restore test
atenea	`moodle`	atenea	2	Moodle 4.5.14 LTS
	`moodle-cron`	atenea	1	same image, `cron.php` every minute
	`moodle-db`	atenea	1	MariaDB 11.4
stats	`grafana`	stats	2	dashboards as code + Infinity plugin (the "?")
	`prometheus`	stats, backup	2	metrics and alert rules, one TSDB per replica
	exporters	stats / global	—	postgres, redis, mysqld, nginx, blackbox, node, cAdvisor

Table 1: Services of the infrastructure.

port 443. Locally the same names are used with `localhost` on port 8080 (browsers resolve `*.localhost` to 127.0.0.1).

3 Images and design decisions

Frontend and cache. `nginx:alpine` with the template feature of the official image (two templates, local and production). Pages of the app are cached for 10 s in nginx (`proxy_cache`, `proxy_cache_lock` and `proxy_cache_use_stale`, so users still get an answer while the app is restarting); static files for one hour. A user that has just posted a review receives the cookie `posted` (10 s) which bypasses the cache, so everybody sees their own review at once. The footer at the bottom left of the page shows *front: <node> / app: <node>*. The name of the node comes from the swarm template `{{.Node.Hostname}}`; the front node is requested by JavaScript to `/__node` (not cached) because the HTML itself may be served from the cache. When the page is refreshed the routing mesh may hand the request to the other frontend replica and the footer changes.

TLS on 443. Production serves HTTPS on 443 with a **self-signed certificate** on `localhost`: nginx cannot start without a certificate, so the entrypoint generates one (key protected with the group passphrase `groupH`, `CERT_PASSWORD`, loaded with `ssl_password_file`) and the browser shows a warning that has to be accepted. The `letsencrypt` service of the diagram is part of the stack and has the full logic (`certbot certonly -webroot`, retries, renewal and the switch of nginx without a restart), but with a local name it stays idle, because a certificate authority can only validate a public domain.

Core application. Python 3.12 slim, non-root, two replicas. The page shows the average, the distribution of ratings, a paginated list and a form with star rating; the average is also published as JSON-LD (schema.org), which the feeder reads. The data of each page is cached in Redis under a key that contains a version number: a new review increments the version, so stale entries are never read again. If Redis is down the page still works, because every cache call is inside `try/except`; if the database is not ready the answer is a clean 503. `/healthz` (liveness) and `/readyz` (database) are used by the healthchecks and by the probes. Writes go to `db:5432` (the PostgreSQL leader) and reads to `db:5433` (the replicas).

PostgreSQL cluster (Patroni). Three PostgreSQL 16 nodes (`pg-1`, `pg-2`, `pg-3`) on app1, app2 and stats, each managed by **Patroni** with a three-member **etcd** (`etcd-1..3`, on the same nodes) as consensus store. Patroni initialises the first node, builds the others as streaming replicas (`pg_basebackup`) and holds the leader key in etcd; if the leader disappears for `ttl` = 30 s a replica takes the key and is promoted, and the old leader is resynchronised with `pg_rewind` when it comes back. Three nodes give a majority for etcd and for the election even if one node is lost, which two nodes could not. `db` is HAProxy (two replicas): it asks the REST API of Patroni which node is the leader (`/primary`) and which are healthy replicas (`/replica`) and exposes one stable address, `db:5432` for the leader and `db:5433` for the replicas. Core, Grafana and the backups use only that name, so a failover needs no change anywhere. Tables and roles are created by the `post_bootstrap` script of Patroni on the first node, not by the application, so several replicas of core starting together cannot collide. Patroni uses a *fast* checkpoint for the base backup of a new replica, otherwise it waits for the spread checkpoint (minutes).

Redis cluster. `redis-1` (master) and `redis-2` (replica) with three Sentinels (app1, app2, stats; quorum 2). The application asks the Sentinels for the master (`redis.sentinel.Sentinel`), so after a failover it finds the new one by itself. The cache is optional: if Redis disappears the application keeps working, only slower.

Moodle. `php:8.3-apache`, release `4.5.14` *pinned* with its SHA-256: the file is downloaded with retries, verified and only then extracted, in its own layer. The entrypoint waits for MariaDB and installs Moodle from the CLI the first time. Only one container installs: a lock directory in the shared moodledata is the mutex and the installer touches a heartbeat file every 10 s; if the heartbeat is older than 90 s the installer died and another container takes the lock (atomic `mv`), empties the half-built schema and starts again. The installation is only trusted once the file `.installed` exists. Sessions are kept in the database, so any replica serves any user; `moodle-cron` is the same image running `cron.php` in a loop.

NFS inside the stack. A swarm service cannot be `privileged`, so instead of the kernel server we use **NFS-Ganesha** in user space, which only needs two capabilities (`DAC_READ_SEARCH`, `SYS_RESOURCE`, `cap_add` of the stack file) and no kernel module. It runs on the atenea node with a local volume and exports `moodledata`, `letsencrypt` and `acme` (NFSv4 only, port 2049). The port is published in `host` mode on that node only and the other nodes mount it at its private address (`NFS_SERVER`); a task that starts before the server is retried by swarm.

Backups and observability. See Sections 7 and 8.

Scope of "production". Production means the complete architecture, not a public service: it runs on our own computer (the seven nodes are simulated with Docker-in-Docker, Section 11.2), it is not opened to the Internet and it has no public domain, so the certificate is self-signed and 443 is served on `localhost`. A deployment on a public server with a real domain would only need `DOMAIN` and `LE_EMAIL`, and certbot would take over; that case is outside this project.

4 Scalability and high availability

4.1 Replicas

The statement asks every service to be scalable, except cron jobs and DBMS. The stateless services run with two replicas (one per node) and can be scaled with `docker service scale`; Prometheus discovers new replicas with the DNS name `tasks.<service>`. The services with one replica are: `letsencrypt`, `backup`, `moodle-cron` and `auto` (periodic jobs: a second copy would only repeat the work), `moodle-db` (DBMS), `nfs` (the shared disk) and `mailpit` (a development tool). PostgreSQL and Redis are DBMS, and are clustered instead of scaled (leader and replicas).

4.2 Failures

Failure	What happens	Recovery
a front node	the routing mesh sends all the traffic to the other `frontend`	automatic
a core / static replica	swarm reschedules; nginx serves `stale` cached pages meanwhile	automatic
Redis master	Sentinels promote the replica (≈4 s on an idle machine; about 30 s when the host was overloaded, because Sentinel enters its `TILT` protection mode); the app keeps working, the cache is optional	automatic
Postgres leader	Patroni elects a new leader among the two replicas and HAProxy follows it; writes fail only during the election (≈3 s if the container is stopped, ≈28 s if it is killed, the `ttl` of the leader key)	automatic
app node (slot 1)	one core left; pg-2 or pg-3 become leader, Redis replica is promoted, etcd keeps its majority (2 of 3)	automatic
stats node	monitoring is down; pg-3 and etcd-3 leave but the quorum holds (2 of 3)	node back
atenea node	Moodle, its database and the NFS (certs) are down; the app keeps working with the certificates it has loaded	node back, or restore from backup
backup node	no new backups; nothing else	node back

Table 2: Failure scenarios.

4.3 Updates

Updates go one replica at a time (`stop-first`, `parallelism: 1`) with `failure_action: rollback`, and are meaningful because the images we build have a `HEALTHCHECK`: a new replica must be healthy before the next one is replaced. The stateless services keep *exactly one replica per node* (`max_replicas_per_node: 1`), so a node failure never takes both; with that restriction there is no free slot to start the new replica before stopping the old one, and the other replica serves meanwhile.

5 Deployment

Local environment (one machine, no TLS):

```
docker compose pull && docker compose up -d
```

Production (seven nodes; `scripts/sim-swarm.sh up` does all this on one computer):

```
# once: build and publish the images
docker login && scripts/publish.sh

# on a manager, after the 7 nodes joined the swarm
scripts/label-nodes.sh # labels the nodes, prints the NFS_SERVER line
export NFS_SERVER=<private IP of atenea>
docker stack deploy -c production.yml lab1
```

All the images are in one public Docker Hub repository, `omarcornejo/lab1grouph`, with one tag per service (`<service>-<version>`), so no registry has to be started before deploying.

6 Access, users and passwords

All the passwords are Docker secrets, files of the `secrets/` folder, and all of them have the value `groupH`, a deliberate simplification of the laboratory (they would be changed before a real deployment). The only exception is the Moodle administrator, because Moodle refuses passwords that do not follow its policy (upper and lower case, digit and symbol).

6.1 Web addresses

Service	Local (`compose.yml`)	Production (`production.yml`)	Login
Reviews application	`http://app.localhost:8080`	`https://app.localhost`	none
Moodle (Atenea)	`http://lms.localhost:8080`	`https://lms.localhost`	yes
Grafana	`http://grafana.localhost:8080`	`https://grafana.localhost`	yes
Mailpit (captured mails)	`http://mail.localhost:8080`	`http://mail.localhost:8080`	none

Table 3: Web addresses. In production the certificate is self-signed, so the browser shows a warning that has to be accepted; port 80 redirects to 443.

Prometheus, Patroni, etcd, HAProxy and the exporters are not published: they are reached from inside the stack (for example, `docker exec` on the container).

6.2 Users and passwords

The reviews application needs no login: anybody can read and write reviews. In both environments the notification mails go to Mailpit, a mail catcher with a web page, so they can be read without any external provider. To send real mail, `RELAY_HOST`, `RELAY_PORT`, `RELAY_USER`, `RELAY_PASSWORD` and `SMTP_TLS` point the relay to a provider.

7 Data management and backups

Databases are not on NFS (locking and network cuts risk corruption). The backup service lives on a node that is neither an app node nor the NFS node, with its own disk, so no live data shares a node with its backup. Every night (03:30) it creates a folder with

- `app.dump.enc`, `grafana.dump.enc`: `pg_dump -Fc` from the *replicas* (`db:5433`, no load on the leader; falls back to the leader), as the read-only role `backup`;
- `moodle.sql.gz.enc`: `mariadb-dump --single-transaction` with a read-only MariaDB account;
- `moodledata.tar.gz.enc`: moodledata without caches and sessions;
- `SHA256SUMS`.

Service	User	Password	Secret file / note
Grafana	admin	groupH	grafana_admin_password
Moodle	admin	groupH-2026	moodle_admin_password
PostgreSQL, administrator	postgres	groupH	pg_admin_password; used only by Patroni
PostgreSQL, application	app	groupH	app_db_password; owner of database app
PostgreSQL, Grafana	grafana	groupH	grafana_db_password; owner of database grafana
PostgreSQL, replication	replicator	groupH	replicator_password
PostgreSQL, backups	backup	groupH	backup_db_password; read only
PostgreSQL, monitoring	exporter	groupH	exporter_db_password; pg_monitor
MariaDB (Moodle)	moodle	groupH	moodle_db_password
MariaDB, backups / monitoring	backup, exporter	groupH	same secrets as in PostgreSQL; read only
Backup encryption	—	groupH	backup_passphrase (AES-256)
Self-signed TLS key	—	groupH	variable CERT_PASSWORD
Feeder token (/api/ratings)	—	groupH	feed_token, bearer token
Docker Hub (pull)	—	—	public repository, no credentials

Table 4: Users and passwords of the project.

Data	Volume	Location
PostgreSQL (3 nodes), etcd	pg-1..3, etcd-1..3	local disk of app1, app2, stats
MariaDB, NFS data	moodle-db, nfs-data	local disk of atenea
moodledata, certificates	moodledata, certs, acme	NFS (atenea)
backups	backups	local disk of the **backup** node
Prometheus (2 replicas)	anonymous volume of each task	local disk of stats and backup
Redis	none	memory (cache, replicated)

Table 5: Where the data is stored.

Every file is encrypted with AES-256-CBC, PBKDF2 (`openssl enc`) and the passphrase of a Docker secret. The folder is written as `.partial` and renamed at the end, so an interrupted backup is never mistaken for a good one. Folders older than 14 days are removed.

Automatic restore test. Every Sunday (and on demand with `docker exec <backup> /restore-test.sh`) the latest backup is verified: checksums, decryption, restoration of the application dump into a throw-away PostgreSQL started inside the container and of the Moodle dump into a throw-away MariaDB, and a count of rows and tables. The result is published as metrics (`backup_restore_test_ok`, rows, tables) that Prometheus scrapes, with alerts if the last backup is older than 25 h or the test fails. The result is the number of reviews and of Moodle tables recovered.

Restoring: `scripts/restore.sh <folder> app|grafana|moodle|moodledata`.

8 Observability

Prometheus uses DNS service discovery (`tasks.<service>`) so scaled replicas are scraped without changes. Table 6 lists the sources.

The dashboard (as code, in git) has rows for overview, containers, nginx, PostgreSQL, Redis, MariaDB/Moodle and backups; the queries use `label_replace` so the same panel works with the swarm and the compose labels of cAdvisor. The Infinity datasource (the "?") shows the average and the ratings captured by the feeder straight from `/api/stats`. Rules in `rules.yml`: target or site down, replication lag, Postgres or Redis without exactly one leader/master, old backup, failed

Source	Watches	What we look at
`core`	the application	requests per replica; reviews posted; cache hits and misses
nginx exporter (inside the frontend image)	each frontend replica	requests per second; active connections
postgres_exporter (one per node) and Patroni	the three PostgreSQL nodes	connections; transactions; replication lag; which node is the leader
redis_exporter	Redis master and replica	role; connected replicas; hit ratio; memory
mysqld_exporter	MariaDB of Moodle	connected threads; queries per second
blackbox_exporter	frontend, core, static, Moodle and Grafana	availability and response time as a user sees them (this is how Moodle is monitored)
cAdvisor and node_exporter	every node	CPU and memory per service and per host
backup service (port 9199)	the backup node	age, size and duration of the last backup; result of the restore test

Table 6: Metric sources.

restore test. Moodle, which has no metrics endpoint, is covered by the probe of its login page and by the MariaDB exporter, which draws the arrow Grafana→atenea of the diagram.

9 Security

- Only nginx is exposed; port 80 serves ACME challenges and redirects to HTTPS (TLS 1.2/1.3, HSTS). Moodle, Grafana and the app are reached only through it.
- **Database roles with least privilege**: `app` owns database `app` and `grafana` owns its own; `replicator` (created by Patroni) can only replicate; `backup` has `pg_read_all_data`; `exporter` has `pg_monitor`; nobody uses the superuser, and `CONNECT` is revoked from `PUBLIC`. MariaDB has separate read-only `exporter` and `backup` accounts.
- Passwords are Docker secrets mounted in `/run/secrets`. All overlay networks are encrypted (IPsec) and each stack is isolated in its own network.
- The SMTP relay is not published; `/metrics` is blocked in nginx. The only other published page is the mail catcher (`mail.localhost:8080`), plain HTTP and without login, which is acceptable in the laboratory because it only holds notification mails.
- Backups are encrypted and verified.

10 Test cases

Each case says what to do, where to look and what must be seen. Commands are run from the project folder (local environment) or on the manager node, `front1` (production, `docker exec front1 ...`).

10.1 Local environment

Table 7: Test cases of the local environment.

	What we do	Where we look	Expected
L1	Publish a review in `app.localhost:8080`	`docker compose logs core | grep "POST /comment"`	status `302`; the review appears at the top and the counter grows

	What we do	Where we look	Expected
L2	(same review)	`docker compose logs smtp | grep status=sent`; page `mail.localhost:8080`	`status=sent`, and the mail "New review (n/5)" is listed
L3	Create a course in Atenea as `admin` (Site administration → Courses)	`docker compose logs moodle | grep "POST /course/edit.php"`; `docker compose exec moodle-db mariadb -umoodle -pgroupH moodle -e "select shortname from mdl_course"`	`303`; the course is in the table
L4	Stop the Postgres leader: `docker compose kill pg-1` (the leader)	`docker compose exec pg-2 patronictl -c /etc/patroni.yml list`; `docker compose logs pg-2 | grep promoted`	another node is *Leader*; "promoted self to leader"; a new review is still accepted
L5	Stop the Redis master: `docker compose stop redis-1`	`docker compose logs sentinel | grep switch-master`	`+switch-master mymaster redis-1 6379 redis-2 6379`; the page keeps working
L6	Backup and restore test	`docker compose exec backup /backup.sh`; `docker compose exec backup /restore-test.sh`	`backup ok: /backups/...`; `restore test ok: n comments, 494 moodle tables`
L7	More replicas: `docker compose up -d -scale core=3`	several requests; header `X-App-Node`	the node name changes between requests
L8	Open Grafana, dashboard *Lab1*	panels of PostgreSQL, Redis, nginx, backups	data in every panel; one Patroni node with value 1

10.2 Production environment

Table 8: Test cases of the production environment.

	What we do	Where we look	Expected
P1	Deploy the stack	`docker node ls`; `docker service ls`; `docker stack ps lab1`	7 nodes *Ready*; every service at its replicas; each task on its node
P2	Open `https://app.localhost`	browser; `curl -k -D- https://app.localhost/`	the page loads after accepting the certificate; `X-Front-Node` and `X-App-Node` alternate between requests
P3	Publish a review	`docker service logs lab1_core | grep "POST /comment"`	`POST /comment ... 302` on one of the two core replicas
P4	(same review)	`http://mail.localhost:8080`; `docker service logs lab1_smtp | grep status=sent`	the mail is listed; `relay=mailpit[...]:1025 ... status=sent`
P5	Create a course in Atenea	`https://lms.localhost`; `docker service logs lab1_moodle | grep course/edit.php`; table `mdl_course` in the `lab1_moodle-db` container	`POST /course/edit.php ... 303`; new row (for example `PRUEBA1`)
P6	Lose a whole node: `docker stop app2`	`docker node ls`; `patronictl list`; `curl -k https://app.localhost/`	`app2` *Down* in ≈13 s; a replica becomes leader; the page answers 200 and a new review is accepted after the election (at most the 30 s `ttl`)
P7	Kill the Postgres leader container: `docker kill <pg-N container>`	`docker service logs lab1_pg-N | grep promoted`; `patronictl list`	"promoted self to leader because I had the session lock"; timeline (TL) grows by one; swarm restarts the container as a replica; no review is lost
P8	Kill the Redis master container	`docker service logs lab1_sentinel-1 | grep switch-master`	`+switch-master mymaster redis-1 6379 redis-2 6379` in about 2 s; the page keeps working
P9	Rolling update: `docker service update -force lab1_frontend`	a request every half second to `https://app.localhost/`	replicas replaced one by one; no request fails

	What we do	Where we look	Expected
P10	Backup on the backup node	`docker exec <lab1_backup container> /backup.sh` and `/restore-test.sh`	`backup ok ... source: db:5433`; `restore test ok`
P11	Restore the application database from a backup	write a review after the backup, then `scripts/restore.sh <folder> app` on the backup node; count the reviews	the review written after the backup disappears and the earlier ones are back; the tables keep their owner and the application keeps working
P12	Open Grafana	`https://grafana.localhost`, dashboard *Lab1*; Prometheus on the stats and backup nodes	36 targets *UP* in each Prometheus; panels with data

11 Results

11.1 Local environment

The stack is started with `docker compose pull` and `docker compose up -d`, and the 29 containers become healthy in under a minute (Moodle takes around three minutes on the first run because of its installation). The application works end to end: a new review is stored in PostgreSQL, shown on the page, sends a mail visible in Mailpit and updates the Grafana dashboards. When the PostgreSQL leader is stopped, another node takes over in about 3 s (about 28 s if the container is killed), and stopping the Redis master promotes its replica. In both cases reviews are still accepted and none are lost. Backups are encrypted, and the automatic restore test recovers the reviews and the 494 Moodle tables.

11.2 Production environment (`production.yml`)

Production runs the same stack on seven nodes, simulated on a single computer with Docker-in-Docker containers (`scripts/sim-swarm.sh`), three of them acting as swarm managers. Every service reaches its replica count and is placed on the node given by its label. The tests gave the following results:

- The three sites answer on port 443 through both front nodes (self-signed certificate), port 80 redirects, and traffic is balanced across front1/front2 and app1/app2.
- When app2 is shut down, swarm detects it in 13 s and a database replica is promoted. The page stays up and reviews are accepted again after the election. When the node comes back, its database rejoins as a replica automatically.
- If one front node fails, the other one serves all the sites.
- A rolling update of the frontend, with a request every 0.5 s, finished with no failed requests.
- Review mails appear in `http://mail.localhost:8080`, courses created in Atenea are stored, and both Prometheus replicas see all their targets up.
- Backups are taken from a database replica on the backup node, encrypted, and pass the restore test.

12 Limitations

- Production is simulated on one machine, so real network latency between nodes was not measured. It is not exposed to the Internet and has no public domain, so the certificate is self-signed (the Let's Encrypt configuration is included but unused).
- PostgreSQL and etcd need two of their three nodes. If two fail together, writes stop until one recovers. The third PostgreSQL node also hosts the monitoring.
- MariaDB, the NFS server and the mail catcher run as single instances, so losing the atenea node stops Moodle until it returns.
- Backups are stored on a single disk with no off-site copy.

- For simplicity, all secrets are `groupH` and are committed to the repository, NFS uses `no_root_squash` (restricted to private networks, its port should be firewalled) and the mail catcher has no authentication.

13 Conclusions

We turned a company infrastructure into a set of containerised services that can be deployed with a couple of commands, either on one machine for development or on seven nodes for production. The system includes the reviews application, Moodle, monitoring, mail and encrypted backups, all published as images.

The hardest part was not getting the services to start but making them tolerate failures: service discovery, leader election and automatic recovery. Many issues only showed up when we deliberately stopped nodes, so testing failures early was essential. In the final system, losing a web server, an application server, the cache or the database leader is transparent to the user, and backups are verified by automatic restores.

The deployment still has the simplifications listed above, but its architecture would be the same on real hardware. Future work would be deploying it on real servers with a public domain, keeping off-site backups and proper secret management.